

# Öğrenci Not Kayıt Sistemi Projesi

Görsel Programlama - Hafta 14

# Öğrenci Not Kayıt Sistemi Projesi: Kapsamlı Uygulama Geliştirme

Dönem boyunca edinilen tüm temel programlama bilgilerini birleştiren bir final projesi hedeflenmektedir.

Bu proje, If-Else, Switch, For, While gibi karar ve döngü yapılarını kapsamaktadır.

Ayrıca TextBox, Button, ListBox gibi temel arayüz (UI) kontrollerinin etkin kullanımını gerektirir.

Projenin temel amacı, veri girişi, işleme, karar verme ve listeleme yeteneklerini tek bir uygulama çatısı altında toplamaktır.



## 7. Hafta (Planın 13. Haftası): Teorik Tekrarın Ana Konuları

Teorik olarak, tüm öğrenilen konuların bir arada kullanılmasını sağlayan bütünleşik bir senaryo oluşturulacaktır.

Kapsanan temel programlama yapıları şunlardır:

- Karar Yapıları (if-else, switch)
- Döngüler (for, while)
- Diziler (Arrays)
- Arayüz Kontrolleri (TextBox, Button, ListBox, ComboBox, Timer)



# Proje Hedefi: Çok Fonksiyonlu Bir Uygulama

Geliştirilecek uygulama, gerçek hayattan bir senaryoyu simüle etmelidir. Bu senaryo, veri girişi (öğrenci bilgileri), veri işleme (ortalama hesaplama), karar yapıları (harf notu atama) ve listeleme (sonuçları gösterme) aşamalarını içerecektir. Bu tür uygulamalar, yazılım mühendisliğinde temel bir gereklilik olan veri yönetimine giriş niteliğindedir.



# Örnek Proje: Öğrenci Not Kayıt Sistemi

Seçilen örnek proje, 'Öğrenci Not Kayıt Sistemi'dir.

Bu sistem, bir CRUD (Create, Read, Update, Delete) benzeri uygulama yapma amacını taşır.

Ancak bu aşamada, diziler ve ListBox ile Silme (Delete) ve Güncelleme (Update) işlemleri karmaşık olacağından, projede öncelikli olarak Ekleme (Create) ve Listeleme (Read) fonksiyonlarına odaklanılacaktır.

# CRUD Kavramı: Veri Yönetiminin Temelleri

CRUD, veritabanı etkileşimli uygulamaların temelini oluşturan dört temel işlemin kısaltmasıdır:

1. Create (Ekleme): Yeni veri kaydı oluşturma (Bizim projemizde yeni öğrenci ekleme).
2. Read (Okuma): Kayıtlı veriyi alma veya listeleme (Bizim projemizde sonuçları ListBox'a listeleme).
3. Update (Güncelleme): Mevcut veriyi değiştirme.
4. Delete (Silme): Mevcut bir veriyi sistemden kaldırma.



# Neden Diziler Kullanılıyor?

Öğrenci verilerini saklamak için programlamada diziler (Arrays) kullanılır. Diziler, aynı türden birden fazla veriyi sabit bir boyutta ve ardışık bellek konumlarında tutmak için idealdir.

Dizilerin kullanımı, not kayıt sisteminin temel yapısının basit ve anlaşılır olmasını sağlar.

Bu yaklaşım, koleksiyon yönetiminin temelini anlamak için kritik öneme sahiptir.



# Arayüz (UI) Tasarımı: Kullanıcı Giriş Alanları

Kullanıcıdan veri almak için üç temel TextBox kontrolü tanımlanmıştır:

1. TextBox (Ad Soyad): Öğrencinin isim bilgisini metin olarak almak için.
2. TextBox (Vize Notu): Vize puanını sayısal olarak almak için.
3. TextBox (Final Notu): Final puanını sayısal olarak almak için.

Bu kontroller, veri girişinin hızını ve doğruluğunu sağlamak için dikkatli tasarlanmalıdır.





# Arayüz (UI) Tasarımı: İşlem Butonları

Uygulamada iki temel eylemi gerçekleştirmek için Buton (Button) kontrolleri kullanılır:

1. Button ('Ekle'): Yeni öğrenci verilerini dizilere kaydetme işlemini başlatır.
2. Button ('Hesapla ve Listele'): Kayıtlı tüm öğrencilerin notlarını işler ve sonuçları ekrana yansıtır.

Bu butonlar, 'Click' olayına bağlı olarak kod akışını yönlendirir.



# Arayüz (UI) Tasarımı: Sonuç ve Raporlama Alanları

Sonuçların kullanıcıya sunulması için iki ana kontrol kullanılır:

1. ListBox: Öğrencileri ve hesaplanan bireysel sonuçları (Ortalama, Harf Notu) listelemek için kullanılır.
  2. Label: Genel istatistikleri (Genel ortalama, kalan/geçen sayısı vb.) göstermek için ayrılmıştır.
- ListBox, verilerin sıralı ve okunabilir bir formatta sunulmasını sağlar.



# Global Kapsamda Dizilerin Tanımlanması

Projenin tüm fonksiyonlarından erişilebilmesi için diziler, global kapsamda tanımlanmalıdır.

Bu tanımlama, dizilerin ömrünün (scope) uygulamanın çalıştığı sürece devam etmesini sağlar.

**\*\*Kapasite:\*\*** Her bir dizi, maksimum 50 öğrenci kaydı tutacak şekilde 'new string/int/double' ile oluşturulur.

# Öğrenci Bilgileri İçin Diziler

İsim ve Soyisimler için string dizisi kullanılır:

```
'string[] ogrenciAdlari = new string;'
```

Vize ve Final notları tam sayı (integer) olarak tutulur:

```
'int[] vizeler = new int;'
```

```
'int[] finaller = new int;'
```

Bu ayırım, veri tipi tutarlılığını sağlamak için zorunludur.

# Hesaplanmış Sonuçlar İçin Diziler

Ortalama, ondalık hassasiyet gerektirdiği için double dizisi kullanılır:

```
'double[] ortalamalar = new double;'
```

Harf notları (AA, BA, CC vb.) metinsel ifadeler olduğundan string dizisi olarak saklanır:

```
'string[] harfNotlari = new string;'
```

# Index Değişkeni: Dinamik Veri Takibi

Sabit boyutlu dizilerde kaç adet geçerli kayıt olduğunu takip etmek için bir 'index' değişkeni kullanılır.

'int index = 0;' olarak başlar.

Bu değişken, dizideki bir sonraki boş konumu işaret eder.

Ekleme işleminden sonra index değeri artırılarak (index++), yeni kaydın bir sonraki konuma yapılacağı belirlenir.

# Ekleme İşlemi (CREATE): Kod Akışının Başlangıcı

Kullanıcı 'Ekle' butonuna tıkladığında, ilgili olay işleyici (event handler) çalışır. İlk adım, kullanıcı arayüzündeki (TextBox'lar) verilerin okunmasıdır.

**\*\*TextBox'lardan veri alımı:\*\***

```
'ogrenciAdlari[index] = txtAd.Text;
```

# Veri Tipi Dönüşümü (Type Casting)

TextBox'lar her zaman metin (string) verisi döndürür.  
Ancak notlar sayısal (int) olarak işlenmeli ve saklanmalıdır.  
Bu nedenle, not verileri dizilere kaydedilmeden önce tip dönüşümü yapılmalıdır:

```
'vizeler[index] = Convert.ToInt32(txtVize.Text);'  
'finaler[index] = Convert.ToInt32(txtFinal.Text);'
```

'Convert.ToInt32' metodu, dönüşüm için standart bir yöntemdir.



# Hata Önleme: Savunmacı Programlama (Enrichment)

Üniversite düzeyinde projelerde, kullanıcı girişinin doğru olduğundan emin olmak esastır (Defensive Programming).

Eğer kullanıcı not alanına sayısal olmayan bir değer girerse, 'Convert.ToInt32' metodu hata (Exception) fırlatır.

Bu tür hataları yönetmek için 'TryParse' veya 'Try-Catch' blokları kullanılmalıdır. Aksi takdirde, uygulama beklenmedik şekilde çökebilir (Runtime Error).



# Ortalama Hesaplama Formülü

Öğrencinin dönem ortalaması, vize ve final notlarının belirli ağırlıklarla çarpılmasıyla hesaplanır.

**\*\*Ağırlıklar:\*\***

- Vize Notu: %40 ağırlık (0.4)
- Final Notu: %60 ağırlık (0.6)

**\*\*Formül:\*\***

'ortalamlar[index] = vizeler[index] \* 0.4 + finaller[index] \* 0.6;'

# Karar Yapıları: Harf Notu Belirleme

Hesaplanan ortalamaya (double) bağlı olarak, öğrencinin akademik harf notu belirlenir.

Bu işlem, karmaşık koşulları sırayla kontrol eden 'if-else if' yapısı ile gerçekleştirilir.

Bu yapı, bir koşul doğru olduğunda diğer koşulların kontrol edilmesini durdurarak performansı artırır.



# Harf Notu Atama Mantığı (Örnek Senaryo)

Gerçek bir not sisteminde kullanılan geniş bir if-else if yapısı simüle edilir:

- Eğer Ortalama  $\geq 90$  ise, Harf Notu = 'AA'.
- Eğer Ortalama  $\geq 85$  ise, Harf Notu = 'BA'.
- Eğer Ortalama  $\geq 80$  ise, Harf Notu = 'BB'.

... ve bu şekilde en düşük 'FF' notuna kadar devam eder.

Bulunan harf notu, 'harfNotlari[index]' dizisine kaydedilir.

# İndeksin İlerlemesi (index++)

Tüm veriler (Ad, Vize, Final, Ortalama, Harf Notu) mevcut 'index' pozisyonundaki ilgili dizilere kaydedildikten sonra, bu pozisyonun bir sonraki ekleme için serbest bırakılması gerekir. Bu, 'index++' komutu ile sağlanır. index'in artırılması, dizinin kapasitesinin dolup dolmadığını kontrol etmek için de önemlidir (50 kayıttan sonra uyarı verilmesi gerekebilir).

# Kullanıcı Deneyimi (UX): Alanların Temizlenmesi

Bir öğrenci başarıyla eklendikten sonra, kullanıcının yeni bir kayıt girmesini kolaylaştırmak amacıyla TextBox'ların içeriği temizlenir. Bu basit adım, kullanıcı arayüzü kalitesini (UX) önemli ölçüde artırır. Temizleme işlemi, her bir 'txtAd', 'txtVize' ve 'txtFinal' kontrolünün 'Text' özelliğinin boş bir değere atanmasıyla yapılır.

# Listeleme İşlemi (READ): Başlangıç

Kullanıcı 'Hesapla ve Listele' butonuna tıkladığında, kayıtlı tüm öğrenci verileri işlenip ListBox'a aktarılır.

Bu işlemin ilk adımı, ListBox'ın önceki içeriklerden temizlenmesidir.

'listBox1.Items.Clear();' komutu, listeleme alanını sıfırlar.

# Neden ListBox'ı Temizlemeliyiz?

Eğer ListBox temizlenmezse ve kullanıcı butona birden fazla kez basarsa, eski kayıtlar ListBox'ta kalır ve yeni kayıtlarla birlikte tekrar listelenir. Bu durum, kullanıcıya yanlış ve tekrarlayan bilgiler sunar. 'Items.Clear()' kullanımı, her seferinde güncel ve doğru veri setinin gösterilmesini garantiler.



# Listeleme Döngüsü: For Kullanımı

Dizilerde kayıtlı olan öğrencileri tek tek okumak için 'for' döngüsü kullanılır. 'for (int i = 0; i < index; i++)'.

**\*\*Döngü Kuralı:\*\*** Döngü, dizinin maksimum kapasitesi (50) kadar değil, sadece gerçekten eklenmiş öğrenci sayısı ('index' değişkenine kadar) kadar dönmelidir.

Bu, performansı artırır ve boş dizinleri işleme sorununu engeller.

# Listelenecek Satırın Oluşturulması

ListBox'ta her bir öğrenci için tek, anlamlı bir metin satırı gösterilmelidir. Bu satır, farklı dizilerdeki ilgili bilgilerin birleştirilmesiyle oluşturulur.

```
'string satir = ogrenciAdlari[i] + ' | Ort: ' + ortalamalar[i] + ' | Harf: ' +  
harfNotlari[i];'
```

Bu işlem, kullanıcının bilgileri kolayca okumasını sağlayan standart bir raporlama formatıdır.

# ListBox'a Satırların Eklenmesi

Hazırlanan her bir metin satırı, döngü içinde ListBox'a eklenir.

`'listBox1.Items.Add(satir);'`

Döngü, 'i = 0' (ilk öğrenci) değerinden başlayıp 'index - 1' (son öğrenci) değerine kadar çalışır.

Böylece tüm kayıtlı öğrenciler, arayüze dinamik olarak yansıtılmış olur.



# Gelişmiş String Yönetimi (Enrichment)

Öğrenci sayısı arttıkça, arka arkaya kullanılan '+' operatörü (string concatenation) performans sorunlarına yol açabilir.

Akademik ve endüstriyel projelerde, bu durumlarda genellikle String Interpolation veya 'StringBuilder' sınıfı tercih edilir.

'StringBuilder', özellikle döngü içinde çok sayıda birleştirme yapıldığında bellek yönetimini optimize eder ve performansı artırır.



# Genel İstatistiklerin Hesaplanması

Proje hedefinde, genel ortalama ve kalan öğrenci sayısı gibi özet bilgilerin Label alanında gösterilmesi belirtilmiştir.

Bu istatistikler, listeleme döngüsü (for) sırasında veya hemen sonrasında hesaplanmalıdır.

**\*\*Örnek Hesaplamalar:\*\***

- Tüm ortalamaların toplamı / index sayısı (Genel Ortalama).
- Harf Notu 'FF' olan kayıtların sayılması (Kalan Sayısı).

## Özet: 'Ekle' Butonu Akışı

1. TextBox'lardan metin verisini al.
2. Notları sayıya dönüştür (Convert.ToInt32).
3. Ortalama hesapla (0.4 / 0.6 ağırlık).
4. if-else if ile harf notunu belirle.
5. Tüm sonuçları mevcut index'e dizilere kaydet.
6. index'i bir artır (index++).
7. TextBox'ları temizle.

## Özet: 'Hesapla ve Listele' Butonu Akışı

1. ListBox'ı temizle (`listBox1.Items.Clear()`).
2. 'i = 0'dan 'index'e kadar dönen bir 'for' döngüsü başlat.
3. Döngü içinde, mevcut öğrencinin Ad, Ortalama ve Harf Notu bilgilerini string olarak birleştir.
4. Oluşturulan satırı ListBox'a ekle (`listBox1.Items.Add()`).
5. Döngü dışında, genel istatistikleri hesapla ve Label'da göster (Kapsam Hedefi).



# Temel Programlama Tekrarı: if-else

if-else yapısı, programın belirli koşullara bağlı olarak farklı kod bloklarını çalıştırmasını sağlar.

Harf notu ataması gibi tek bir koşul setinden sadece bir sonucun seçilmesi gereken durumlar için idealdir.

Eğer koşullar birbiriyle bağımsız olsaydı, art arda gelen bağımsız 'if' ifadeleri kullanılabilirdi.





# Temel Programlama Tekrarı: switch

Switch yapısı, bir değişkenin değerini birden fazla sabit değerle karşılaştırmak için kullanılır.

Harf notlarının sayısal değerlere dönüştürülerek (A=4.0, B=3.0) akademik puana çevrilmesi gibi durumlarda, if-else if yerine switch daha okunabilir bir alternatif sunabilir.

Ancak ortalama aralıkları kontrol edildiğinde 'if-else if' genellikle daha esnektir.

# Temel Programlama Tekrarı: for Döngüsü

For döngüsü, belirli bir başlangıç değeri, bir bitiş koşulu ve bir artırma/azaltma ifadesi ile tanımlanır.

Listeleme işleminde, dizinin başlangıcından ( $i=0$ ) 'index' değerine kadar kesin olarak bilinen sayıda iterasyon gerektiği için for döngüsü en uygun seçimdir. Bu, dizideki her bir elemanı sırayla işlemek için kontrol mekanizması sağlar.



# Temel Programlama Tekrarı: while Döngüsü

While döngüsü, bir koşul doğru kaldığı sürece kod bloğunu tekrar eder. Öğrenci kayıt sistemi gibi uygulamalarda, while döngüsü genellikle kullanıcıdan doğru giriş alınana kadar tekrar edilmesi gereken veri doğrulama (validation) süreçlerinde kullanılır. Örneğin, notun 0 ile 100 arasında girildiğinden emin olmak için.



# Dizilerin Kritik Rolü ve Kısıtlamaları

Diziler (Arrays), projenin temel veri depolama yapısıdır. En büyük kısıtlamaları, boyutlarının baştan sabitlenmiş olmasıdır (new string). Eğer 50'den fazla öğrenci eklenmeye çalışılırsa, program hata verecek (Index Out of Range Exception). Bu durum, dinamik veri yapılarının (Listeler, Koleksiyonlar) gerekliliğini ortaya koyar (Enrichment).



# Veri Yapıları Alternatifi: Dinamik Listeler (Enrichment)

Üniversite projelerinde, gerçek dünya senaryolarında sabit boyutlu diziler yerine 'List<T>' gibi dinamik koleksiyonlar tercih edilir. Listeler, boyutu otomatik olarak yönetir ve 'index' takibine gerek kalmadan 'Add()' metoduyla kolayca yeni öge eklenmesini sağlar. Bu, ilerideki projeler için daha sürdürülebilir bir yaklaşımdır.



# TextBox Kontrolü Detaylı İnceleme

TextBox, kullanıcının tek bir metin satırı girmesini sağlayan en temel UI bileşenidir.

Önemli Özellikler:

- Text: Kontrolün içerdiği metni okur/yazar (veri alımı buradan yapılır).
- Multiline: Çok satırlı giriş imkanı.
- ReadOnly: Kullanıcının düzenleme yapip yapamayacağını belirler.

# Button Kontrolü Detaylı İnceleme

Button kontrolü, kullanıcı etkileşimi sonucu programda bir eylemi (action) tetiklemek için kullanılır.

**\*\*Temel Olay:\*\*** 'Click' olayıdır.

Kodun büyük bir kısmı, bu butona basıldığında (Ekle, Hesapla ve Listele) çalışacak şekilde yazılır.

Üniversite projelerinde, butonların net isimleri ve amaçları olması kullanıcı için önemlidir.



# ListBox Kontrolü Detaylı İnceleme

ListBox, birden fazla öğeyi dikey bir liste halinde görüntülemek için kullanılır. 'Items' özelliği, listeye veri eklemek veya var olan veriyi temizlemek için temel erişim noktasıdır.

Bu projede, hesaplanmış final sonuçlarını raporlamak amacıyla kullanılır.





# Label Kontrolü Detaylı İnceleme

Label kontrolü, genellikle kullanıcıya bilgi vermek, diğer kontrolleri etiketlemek veya dinamik sonuçları (çıktıları) göstermek için kullanılır. Bu projede, genel ortalama veya kalan/geçen öğrenci sayısı gibi özet istatistikleri göstermek için ayrılmıştır. Label'ın içeriği, kod içinden 'Label.Text' özelliği değiştirilerek güncellenir.

# ComboBox ve Timer Kontrollerinin Konumu

Kaynakta ComboBox ve Timer da genel tekrar konuları arasında listelenmiştir.

**\*\*ComboBox:\*\*** Not sistemine 'Ders Adı' veya 'Dönem' seçimi eklemek için kullanılabilir.

**\*\*Timer:\*\*** Belirli aralıklarla otomatik kaydetme veya güncel saati gösterme gibi zaman tabanlı işlevler için kullanılır.

Bu projede doğrudan zorunlu olmasalar da, kapsamlı bir tekrarın parçasıdırlar.

# Modüler Programlama Yaklaşımı (Enrichment)

Üniversite projelerinde kodun karmaşıklığı arttıkça, işlevleri ayrı fonksiyonlara veya sınıflara bölmek gerekir.

Örneğin, 'Ortalama Hesaplama' ve 'Harf Notu Bulma' mantığı ayrı metotlar içinde tanımlanmalıdır.

Bu, okunabilirliği, test edilebilirliği ve kodun yeniden kullanılabilirliğini artırır.

# Veri Girişinde Tip Çatışması Sorunu

Kullanıcıdan alınan tüm veriler (notlar dahil) başlangıçta metin (string) formatındadır.

Ancak matematiksel işlemler (ortalama hesaplama) sadece sayısal tiplerle (int, double) yapılabilir.

Bu zorunlu dönüşüm, projenin en sık karşılaşılan hata kaynaklarından biridir (Format Exception).



# Convert Sınıfı ve Alternatifleri (Enrichment)

'Convert.ToInt32()', genel bir dönüşüm metodudur ve null değerler üzerinde de çalışabilir.

Alternatif olarak, sayısal tiplerin kendi 'Parse()' metotları vardır (örn: 'Int32.Parse()').

Ancak en güvenli yöntem, hata durumunu kontrol eden 'Int32.TryParse()' kullanmaktır; bu, dönüşüm başarısız olursa bir hata fırlatmaz.



# Mantıksal Hata Yönetimi: Sıfıra Bölme

Eğer sisteme hiç öğrenci eklenmemişse ( $\text{index} = 0$ ) ve genel ortalama hesaplanmaya çalışılırsa, toplam ortalamamanın 'index'e bölünmesi sıfıra bölme hatasına (Divide By Zero Exception) yol açar. Bunu önlemek için listeleme ve genel hesaplama işlemlerinden önce mutlaka 'if ( $\text{index} > 0$ )' kontrolü yapılmalıdır.



# Ağırlıklı Ortalama Mantığı

Vize (%40) ve Final (%60) ağırlıkları, dersin zorluk derecesini ve sınavların önemini yansıtır.

Final notunun vizeden daha yüksek bir etkiye sahip olması ( $0.6 > 0.4$ ), genellikle final sınavının daha kapsamlı ve belirleyici olduğunu gösterir. Bu, üniversite notlandırma sistemlerinin standart bir uygulamasıdır.



# Harf Notu Belirlemede Koşul Sıralaması

if-else if merdiveni kullanılırken, koşullar doğru sırayla kontrol edilmelidir. Genellikle, en kısıtlayıcı koşuldan (en yüksek not, örn: Ortalama  $\geq 90$ ) en geniş koşula (en düşük not, örn: Ortalama  $< 60$ ) doğru ilerlemek gerekir. Aksi takdirde, yanlış bir koşulun erken yakalanması sonucu hatalı harf notları atanabilir.





# index Değişkeninin Çift Görevi

'index' değişkeni, sadece yeni verinin ekleneceği boş pozisyonu göstermekle kalmaz.

Aynı zamanda, o ana kadar kaç tane başarılı kayıt yapıldığının da sayacıdır. Bu, listeleme döngüsünün 'index'i bitiş koşulu olarak kullanmasının ana nedenidir (sadece eklenen öğrenci sayısı kadar dön).



# Veri Kalıcılığı Sorunu (Enrichment)

Dizilerde (Arrays) saklanan veriler, uygulama kapatıldığında kaybolur. Bu, üniversite projelerinde 'Veri Kalıcılığı' (Persistence) ihtiyacını doğurur. Daha gelişmiş sistemler, verileri tutmak için Dosya İşlemleri (metin veya ikili dosyalar) veya SQL, NoSQL gibi Veritabanı Sistemleri kullanır.



# Kapsam Geniřletme: Güncelleme (UPDATE) İřlemi

Bir öđrencinin notunu deđiřtirmek (Güncelleme), mevcut dizilerde daha karmařıktır.

**\*\*Gerekli Adımlar:\*\***

1. ListBox'tan güncellenecek öđrenciyi seçmek (Index'i bulmak).
2. Seçilen index'teki notları yeni deđerlerle deđiřtirmek.
3. Ortalama ve Harf Notunu yeniden hesaplamak.
4. ListBox'ı güncel verilerle yenilemek.

# Kapsam Geniřletme: Silme (DELETE) iřlemi

Dizilerden bir kaydı silmek, sadece ilgili hücreyi boş bırakmak anlamına gelmez; dizide bir 'boşluk' yaratır. Silme işlemi için, silinen kaydın ardından gelen tüm kayıtların bir adım öne kaydırılması gerekir (shifting). Bu manuel kaydırma işlemi, dizileri CRUD'un silme aşaması için zahmetli kılar.

# İleri CRUD: Nesne Tabanlı Yaklaşım (Enrichment)

Daha temiz ve yönetilebilir kod için, her bir veri parçasını ayrı dizilerde tutmak yerine (ogrenciAdlari, vizeler, finaller),  
bir 'Ogrenci' sınıfı (Object-Oriented Programming) tanımlanmalıdır.  
Bu, Ad, Vize, Final gibi tüm bilgileri tek bir nesne içinde gruplar.  
Böylece tek bir 'List<Ogrenci>' koleksiyonu, tüm sistemi yönetebilir.

# Hata Ayıklama (Debugging) Teknikleri

Üniversite projelerinin geliştirilmesinde hata ayıklama becerisi kritiktir. Programın akışını izlemek için 'Breakpoint'ler (Kesme Noktaları) kullanılır. Özellikle 'Convert.ToInt32' çağırısı, not hesaplama formülü ve 'for' döngüsünün sınırları gibi karmaşık noktalarda hataların nerede oluştuğu bu teknikle tespit edilebilir.

# Kullanılabilirlik (Usability) İpuçları

Yüksek kaliteli bir üniversite projesi, sadece çalışmakla kalmaz, aynı zamanda sezgisel olmalıdır.

- **\*\*Geri Bildirim:\*\*** 'Ekle' işleminden sonra kullanıcıya başarılı bir mesaj göstermek.
- **\*\*Hata Mesajları:\*\*** Geçersiz veri girişi durumunda (örn: harf yerine not girmek) net ve kullanıcı dostu uyarılar sunmak.
- **\*\*Sekme Düzeni:\*\*** Veri giriş alanları arasında Tab tuşu ile kolayca geçiş yapma.



# Proje Performansını Artırma (Enrichment)

Mevcut sistem 50 öğrenciyle sınırlıdır. Ancak binlerce kayıt olması durumunda, listeleme döngüsünün (for loop) çok hızlı çalışması gerekir. Bu durumda, ListBox yerine sanallaştırılmış (virtualized) listeleme kontrolleri kullanmak veya verileri bellekte optimize edilmiş koleksiyonlarda tutmak performans kazandırır.





# Vize ve Final Notu Alma: 'TextBox.Text'

Kullanıcı arayüzünden notları programa aktarmak için, ilgili TextBox kontrolünün 'Text' özelliğine erişilir.

Örnek: 'txtVize.Text' veya 'txtFinal.Text'.

Unutulmamalıdır ki, bu değer bir stringdir ve matematiksel işlem öncesinde dönüştürülmelidir.



# Harf Notu Atama: Detaylı Kontrol Senaryoları

Harf notu mantığında, belirlenen bir minimum ortalama (örn: 60) genellikle geçme/kalma sınırını belirler.

'if-else if' yapısının en son 'else' bloğu, genellikle belirlenen tüm şartları sağlamayanlar için kullanılır (örn: Harf Notu = 'FF').

Listeleme (READ) sırasında bu 'FF' notları sayılarak kalan öğrenci istatistiği Label alanına yansıtılabilir.

# Proje Geliştirme Yaşam Döngüsü (SDLC) (Enrichment)

Bu proje, bir yazılım geliştirme yaşam döngüsünün (SDLC) Uygulama ve Test aşamalarına karşılık gelir.

Üniversite projelerinde, kodlamaya başlamadan önce arayüzün ve veri yapılarının (diziler/index) net bir şekilde planlanması kritiktir.

Kaynak metin, bu planlamanın sonucunda ortaya çıkan kod akışını ve arayüz bileşenlerini listeler.



# Kontrol Adları ve Tutarlılık

Profesyonel yazılım geliştirmede, kontrollere anlamlı ve tutarlı isimler verilmesi önemlidir (örn: 'txtAd', 'listBox1').

Bu projedeki adlandırma (txtVize, txtFinal), hangi TextBox'ın hangi veriyi tuttuğunu açıkça belirtir.

Bu, kod okunabilirliğini artırır ve uzun vadede hataları azaltır.



# Dizilerin Sıfır Tabanlı İndekslemesi

Programlamada, dizilerdeki ilk eleman her zaman 0 (sıfır) indeksine sahiptir. Bu nedenle, 'index = 0' başlangıç değeri, ilk kaydın buraya yapılacağını gösterir.

Listeleme döngüsü de 'for (int i = 0; ...)' ile başlar, bu kurala uyar.

# Bellekteki Dizilerin Yapısı (Enrichment)

Çoğu modern dilde (C# gibi), diziler referans tipindedir. Bu, 'ogrenciAdlari' değişkeninin doğrudan veriyi değil, verinin bellekte nerede başladığını gösteren bir adresi tuttuğu anlamına gelir. Bu kavram, büyük veri setleri veya koleksiyonlar yönetilirken bellek verimliliği açısından kritiktir.

# TextBox Temizleme Yöntemleri

TextBox'ı temizlemek için en yaygın kullanılan yöntem, 'Text' özelliğini boş bir string'e atamaktır.

Bazı dillerde 'Clear()' metodu da bulunabilir, ancak amaç her zaman kullanıcıya temiz bir giriş alanı sunmaktır.



# Döngü Sınırının Kritikliği

Listeleme döngüsünde ' $i < \text{index}$ ' koşulunun kullanılması hayati öneme sahiptir. Eğer ' $i < 50$ ' (dizi kapasitesi) kullanılsaydı ve sadece 5 öğrenci eklenmişse ( $\text{index}=5$ ), döngü geri kalan boş 45 pozisyonu okumaya çalışacak ve program hata verecekti (boş dizinlere erişim).





# Dizilerde Farklı Tiplerin Eşleştirilmesi

Bu projede beş farklı dizi kullanılmıştır (Adlar, Vizeler, Finaller, Ortalamalar, Harf Notları).

Bu dizilerin hepsi aynı 'index' değişkenini paylaşır.

'index=3'teki öğrencinin adı, vizesi ve ortalaması bu beş dizideki 3. pozisyonlar tarafından tanımlanır.

Bu yapıya 'Paralel Diziler' denir ve nesne tabanlı programlamadan önceki yaygın bir modeldir.



# Hata Durumu: Boş Veri Girişi

Kullanıcı bir TextBox'ı boş bırakıp 'Ekle'ye basarsa:

- 'txtAd.Text' boş string olarak dizine kaydedilir (sorun yok).
- 'Convert.ToInt32(txtVize.Text)' boş stringi sayıya dönüştüremeyeceği için hata fırlatır.

Bu, ekleme işlemi öncesinde 'if (string.IsNullOrEmpty(txtVize.Text))' gibi kontrolleri zorunlu kılar (Enrichment).

# Kapanış: Tüm Konuların Birlikteliği

Öğrenci Not Kayıt Sistemi, if-else, for, diziler ve UI kontrollerinin bir arada, uyum içinde çalıştığı bütünleşik bir yapı sunar.  
Bu tür projeler, üniversite öğrencilerinin teorik bilgiyi pratik yazılım geliştirmeye dönüştürme becerisini ölçer.  
Dönem boyunca öğrenilen tüm yetkinliklerin uygulamaya dökülmesini sağlar.



# İleri Adımlar: İlişkisel Veritabanı Modelleri (Enrichment)

Gerçek dünyadaki kayıt sistemleri, verileri kalıcı olarak saklamak ve daha karmaşık ilişkileri yönetmek için SQL veritabanlarını kullanır. Bu sistemde oluşturulan her dizi, bir veritabanı tablosundaki bir sütunu temsil eder. Öğrenci Not Kayıt Sistemi projesi, öğrencileri bu tablo ve sütun mantığına hazırlar.



# İleri Adımlar: Kullanıcı Yetkilendirme (Enrichment)

Daha gelişmiş bir not kayıt sistemi, farklı kullanıcı rolleri gerektirir.  
Örn: Öğrenci (sadece notlarını görebilir), Öğretim Üyesi (notları ekleyebilir/güncelleyebilir), Yönetici (tüm verilere erişebilir).  
Bu, giriş ekranı (Login UI) ve yetkilendirme (Authorization) mekanizmalarının eklenmesini gerektirir.



# Kapsamlı Uygulama Geliştirme Prensipleri

Kapsamlı bir uygulama geliştirirken, kaynakta belirtilen tüm bileşenlerin belirli bir plan dahilinde entegre edilmesi gerekir.

1. Planlama (Veri yapıları ve UI tasarımı).
  2. Kodlama (Olay işleyicileri ve mantık).
  3. Test Etme (Tüm senaryoların denenmesi).
- Bu aşamalar, projenin başarısını belirler.



# Veri İşleme Hızı ve Etkinlik

Dizilere erişim, genellikle programlamada en hızlı veri erişim yöntemlerinden biridir (indeksleme sayesinde).

Ancak listeleme döngüsünde, özellikle döngü içinde gereksiz hesaplamalar veya string manipülasyonları yapmaktan kaçınılmalıdır.

Satır oluşturma işlemi sadece ListBox'a eklenecek çıktıyı hazırlamalıdır, performansı düşürecek başka bir yük taşımamalıdır.



# Kullanıcı Geri Bildirim Mekanizmaları

Ekleme başarılı olduğunda veya bir hata oluştuğunda, arayüzde bir 'Durum Etiketi' (Status Label) kullanılabilir.

Bu, kullanıcıya işlemin sonucunu bildirir (örn: 'Öğrenci başarıyla kaydedildi' veya 'Hata: Lütfen notları kontrol edin').

Bu küçük detay, projenin profesyonel kalitesini artırır.





# Proje Çıktısı: Raporlama Standardı

ListBox'ta gösterilen çıktı formatı (örn: Ad | Ort: X | Harf: Y), verinin kolayca anlaşılabilmesi için önemlidir.

Raporlama aşamasında, verilerin hizalanması (alignment) veya renk kodlaması gibi görsel iyileştirmeler (Enrichment) yapılabilir.

Bu, özellikle sonuçların karşılaştırılması gerektiğinde faydalıdır.



# İleri Konular: Birim Testleri (Enrichment)

Üniversite düzeyinde projelerde, fonksiyonların doğru çalıştığını kanıtlamak için birim testleri (Unit Tests) yazılmalıdır.

Örn: Ortalama hesaplama fonksiyonuna (0.4/0.6 ağırlık) belirli girişler verilerek, beklenen harf notunun doğru dönüp dönmediği test edilir.

Bu, projenin güvenilirliğini kanıtlamanın en iyi yoludur.



# Kapanış Özeti: Veri Yönetimi Temelleri

Bu proje, temel programlama yapılarını kullanarak dinamik veri yönetimi yapmanın basit ama kapsamlı bir örneğidir. Dizilerle 'index' takibi, veri dönüşümü, koşullu mantık ve arayüz etkileşimleri (TextBox, Button, ListBox), modern yazılımların iskeletini oluşturur.



# Sistem Sınırlamaları ve Ölçeklenebilirlik

Dizi boyutunun 50 ile sabitlenmesi, projenin basitliğini korur. Ancak ölçeklenebilirlik (scalability) için dinamik yapılar (Listeler) veya veritabanı entegrasyonu zorunludur. Üniversite projesinde, 50 kişilik sınırın dolması durumunda kullanıcıya bilgilendirici bir uyarı gösterilmelidir (Enrichment).



# Gelecek Projeler İçin Temel

Bu proje ile kazanılan deneyimler, ileri düzey derslerde Veritabanı Programlama, Web Uygulama Geliştirme ve Mobil Uygulama Geliştirme gibi konularda temel oluşturur.  
Özellikle CRUD döngüsünü kavrama yeteneği, yazılım kariyerinin temel taşıdır.



# Öğrenci Not Kayıt Sistemi Projesi

Süleyman **EZDEMİR**

*suleyman.ezdemir@giresun.edu.tr*

